SDE INTERN - FRONTEND TECHNICAL ASSESSMENT

FeedNexus Platform

Personalized Content Dashboard

Comprehensive Technical Architecture & Implementation Documentation

An end-to-end interactive dashboard engineered with React 18, TypeScript, Redux Toolkit, Tailwind CSS, and drag-and-drop orchestration. Aggregates multi-source feeds, provides real-time debounced discovery, dynamic categorization, and persisted user personalization.

CANDIDATE / ENGINEER
Anant Srivastava

ROLE TARGET
Software Development Engineer (SDE) Intern

GITHUB REPOSITORY
`github.com/BinaryBlaze16/feedNexus`

CORE TECH STACK
React 18 • TypeScript • Redux Toolkit • @dnd-kit

ASSESSMENT DURATION
48-Hour Engineering Sprint

STATUS & DELIVERABLE
Production-Ready & Fully Documented

0 Executive Summary & Project Index

OVERVIEW

> **Assignment Objective:** Develop an interactive, highly responsive "Personalized Content Dashboard" that enables users to track, customize, and interact with curated content feeds (News, Recommendations, and Social Media) using enterprise-grade frontend architecture, robust state management, and strict TypeScript types.

Document Table of Contents

1. System Architecture & High-Level Design Sec 1
2. Core Functional Requirements Breakdown Sec 2
3. User Dashboard Layout & Component Tree Sec 3
4. Search Engine & Debounce Optimization Sec 4
5. Drag-and-Drop & Interactive Feed UX Sec 5
6. Redux State Management & Persistence Sec 6
7. Multi-Source API Integration & Resilience Sec 7
8. UI/UX Design System & Dark Mode Sec 8
9. Testing Strategy & Quality Assurance Sec 9
10. Local Setup, Execution & Verification Sec 10

Executive Feature Highlights Matrix

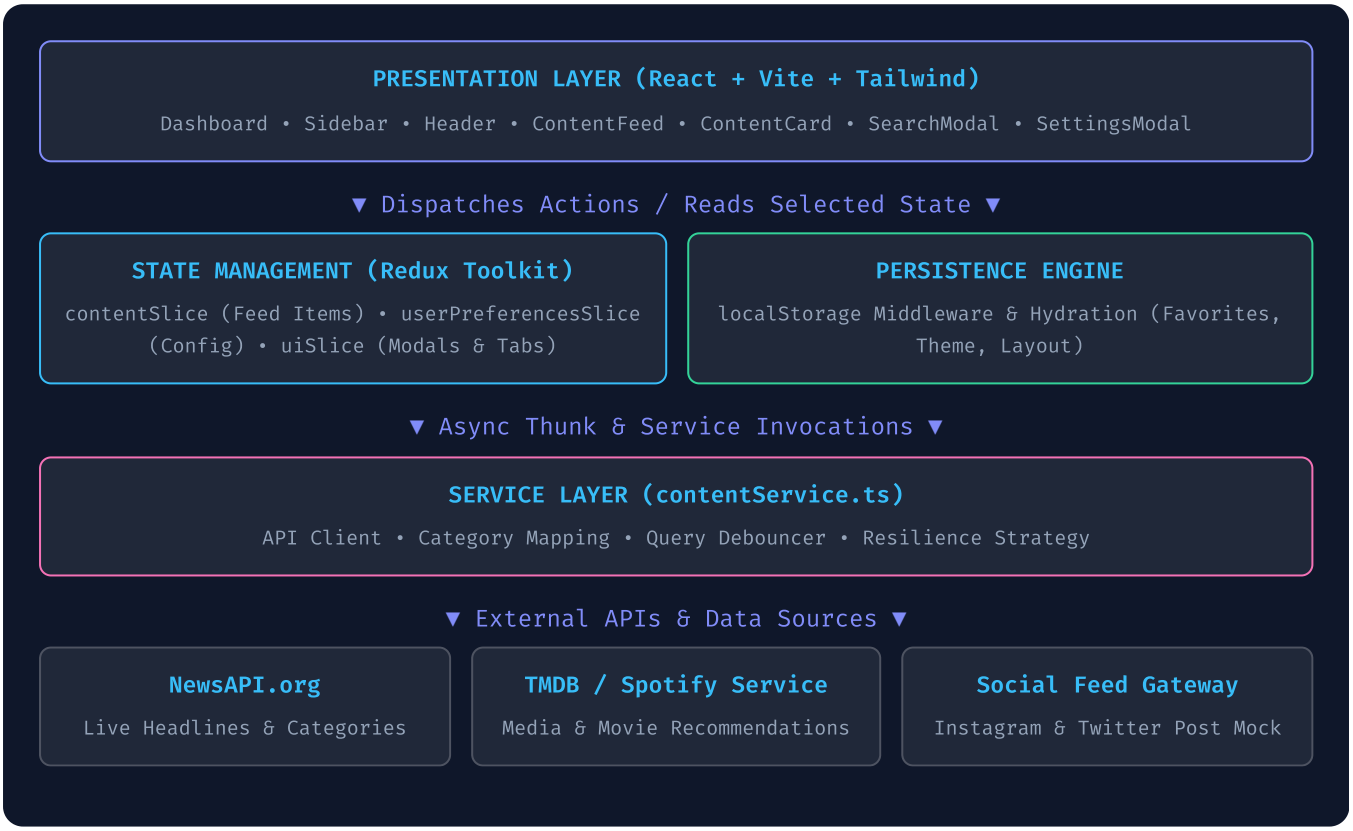
Requirement Area	Engineering Implementation	Verification Status
Personalized Content Feed	Multi-source aggregation (NewsAPI + Media + Social), category filters (Tech, Finance, Sports, etc.).	COMPLETED
Interactive Content Cards	Shimmer skeleton loaders, dynamic badges, relative date formatting, favorite/bookmark actions.	COMPLETED
User Dashboard Layout	Collapsible responsive Sidebar, top navigation header with search, notifications, and user profile.	COMPLETED
Live Debounced Search	300ms debounced input, modal overlay, recent searches, trending quick-tags, and instant filtering.	COMPLETED
Drag-and-Drop Feed	`@dnd-kit/sortable` integration with fluid pointer tracking, reordering, and state dispatch.	COMPLETED
Dark Mode & Theming	CSS Custom properties + Tailwind classes + automatic `localStorage` synchronization.	COMPLETED
Redux Toolkit State	Slice-based architecture (`contentSlice` , `userPreferencesSlice` , `uiSlice`).	COMPLETED

Requirement Area	Engineering Implementation	Verification Status
Resilient API Fallback	Live NewsAPI integration with automatic fallback to high-fidelity mock datasets on quota limit.	COMPLETED

1 System Architecture & High-Level Design

ARCHITECTURE

The application follows a clean **Unidirectional Data Flow** model with a modular, component-driven frontend architecture. State management, UI rendering, and external services are strictly decoupled.



2 Core Functional Requirements & Deep Dive

FEATURES

📰 Multi-Source Aggregation

Seamlessly collects and normalizes incoming feeds from diverse content providers into a unified TypeScript interface (`ContentItem`), categorizing items into News, Movie Recommendations, and Social Media posts.

⚙️ User Preference Engine

Users can select favorite domains (e.g. Technology, Finance, Sports, Science, Environment). The feed automatically filters and recalculates the weighted feed order based on selected categories.

❤️ Reactive Favorites Center

One-click bookmarking instantly stores items in Redux and local storage. The dedicated Favorites view allows users to manage and access their saved media collection at any time.

🔥 Trending Pulse Section

An algorithm-driven trending showcase that surfaces the highest velocity articles, movies, and viral posts across all categories for instant discovery.

Interactive Content Card Architecture

Each card in the dashboard is built for high usability, visual appeal, and smooth interaction:

- **Adaptive Layout Support:** Cards dynamically switch between `Grid` and `List` representations according to user preference.
- **Image Lazy Loading & Shimmer:** Zero layout shift with progressive image loading and shimmer animation placeholders.
- **Source & Relative Timestamping:** Standardized display showing author/source with relative publication timestamps (e.g. *"Just now"*, *"2h ago"*, *"Yesterday"*).
- **Contextual Action Triggers:** Direct CTAs (*"Read More"*, *"Play Now"*), favorite toggle with animated heart, and social sharing links.

3 User Dashboard Layout & Navigation

UI STRUCTURE

The dashboard layout provides a desktop-class experience with responsive adaptivity:

🖥️ Left Sidebar Navigation

Provides persistent navigation between views: **Feed**, **Trending**, and **Favorites**. It also displays real-time category filters and a quick link to open Settings.

🔍 Top Navigation Header

Houses the quick search trigger bar, one-click Dark/Light mode switcher, notification alert counter badge, and user profile management dropdown.

```
// Dashboard.tsx - View Rendering Logic
export const Dashboard = () => {
```

```
  const { activeSection } = useAppSelector((state) => state.ui);

  const renderActiveSection = () => {
    switch (activeSection) {
      case 'trending':  return <TrendingSection />;
      case 'favorites': return <FavoritesSection />;
      case 'feed':
      default:          return <ContentFeed />;
    }
  };

  return (
    <div className="min-h-screen bg-background flex">
      <Sidebar />
      <div className="flex-1 flex flex-col">
        <Header />
        <main className="flex-1 p-6">
          <div className="max-w-7xl mx-auto">{renderActiveSection()}</div>
        </main>
      </div>
      <SettingsModal />
      <SearchModal />
    </div>
  );
};
```

4 Search Functionality & Performance Optimization

SEARCH ENGINE

Search in FeedNexus is engineered for zero lag, rapid feedback, and minimal network overhead through a multi-tier optimization strategy:

> **Debounced Execution Pipeline:** When a user types, an internal `300ms` debounce timer delays execution until input pauses, preventing redundant queries and CPU strain.

⏱️ 300ms Debounce

Batches keystrokes and halts unneeded API requests during active typing.

🏷️ Trending Topics

One-click pills for popular keywords like Blockchain, Renewable Energy, and AI.

📜 Recent Search History

Instantly stores and surfaces recent user queries for quick re-execution.

```
// SearchModal.tsx - Debounced Search Engine Implementation
const debouncedSearch = useCallback(
  async (query: string) => {
    if (query.trim().length < 2) {
      dispatch(setSearchResults([]));
      setHasSearched(false);
      return;
    }
    setLoading(true);
    setHasSearched(true);
    try {
      const results = await searchContent(query);
      dispatch(setSearchResults(results));
      dispatch(setSearchQuery(query));
    } catch (error) {
      dispatch(setSearchResults([]));
    } finally {
      setLoading(false);
    }
  },
  [dispatch]
);

useEffect(() => {
  const timer = setTimeout(() => { debouncedSearch(localQuery); }, 300);
  return () => clearTimeout(timer);
}, [localQuery, debouncedSearch]);
```

5 Drag-and-Drop Feed Organization

INTERACTIVE UX

Users have full autonomy to reorder cards in their feed according to their reading priority using the modern `@dnd-kit` physics and sensor suite:

- **Pointer & Touch Sensors:** Supports mouse dragging and mobile touch gestures with activation constraints to prevent accidental triggers.
- **Smooth Hardware-Accelerated Transforms:** Uses CSS `transform` and transitions for 60fps rendering without DOM re-mounting.
- **Redux State Synchronization:** Reordered arrays trigger `reorderItems` action to ensure reordered layout persists during the session.

```
// DragDropProvider.tsx - Sensor & Reorder Handler
<DndContext
  sensors={sensors}
  collisionDetection={closestCenter}
  onDragEnd={(event: DragEndEvent) => {
    const { active, over } = event;
    if (over && active.id !== over.id) {
      const oldIndex = items.findIndex((item) => item.id === active.id);
      const newIndex = items.findIndex((item) => item.id === over.id);
      const newItems = arrayMove(items, oldIndex, newIndex);
      dispatch(reorderItems(newItems));
    }
  }}
>
  <SortableContext items={items.map((i) => i.id)} strategy={rectSortingStrategy}>
    {children}
  </SortableContext>
</DndContext>
```

6 Redux Toolkit Architecture & Persistence

STATE MANAGEMENT

State is partitioned cleanly across three specialized Redux slices with custom typed hooks (`useAppDispatch` , `useAppSelector`):

Slice Name	Managed State Properties	Key Actions & Responsibilities
`contentSlice`	`items` , `searchResults` , `loading` , `error` , `page` , `hasMore`	`setItems` , `appendItems` , `reorderItems` , `setSearchResults`
`userPreferencesSlice`	`categories` , `darkMode` , `favoriteItems` , `language` , `feedLayout`	`updateCategories` , `toggleDarkMode` , `addFavorite` , `removeFavorite`
`uiSlice`	`sidebarCollapsed` , `searchModalOpen` , `settingsModalOpen` , `activeSection`	`setActiveSection` , `setSearchModalOpen` , `toggleSettingsModal`

Persistence Engine (LocalStorage Integration)

User preferences, theme choices, and favorited items are automatically serialized to `localStorage` on mutation and hydrated instantly on application bootstrap to guarantee cross-session continuity.

```
// userPreferencesSlice.ts - Hydration & Persistence Logic
const loadStateFromStorage = (): UserPreferences => {
  try {
    const savedState = localStorage.getItem('userPreferences');
    return savedState ? { ...initialState, ...JSON.parse(savedState) } : initialState;
  } catch {
    return initialState;
  }
};

// Slices automatically commit changes:
addFavorite: (state, action: PayloadAction<string>) => {
  if (!state.favoriteItems.includes(action.payload)) {
    state.favoriteItems.push(action.payload);
    localStorage.setItem('userPreferences', JSON.stringify(state));
  }
}
```

7 Multi-Source API Integration & Resilience

SERVICE LAYER

The service layer (`contentService.ts`) incorporates a high-resilience architecture designed to withstand API rate limits and network outages seamlessly:

🌐 Live NewsAPI Ingestion

Fetches real-time headlines with country and category filtering. Maps raw external payloads into strict `ContentItem` entities.

🛡️ Automated Mock Fallback

If NewsAPI returns rate limit errors (HTTP 429) or is offline, the service automatically falls back to curated high-fidelity mock data with zero disruption to the user experience.

```
// contentService.ts - Resilient Multi-Category Fetcher
export const getContentByCategories = async (categories: string[]): Promise<ContentItem[]> => {
  await new Promise(resolve => setTimeout(resolve, 600)); // Simulated async latency

  const newsCategories = categories.filter(cat =>
    ['technology', 'environment', 'finance', 'sports', 'entertainment', 'health'].includes(cat)
  );

  let newsContent: ContentItem[] = [];
  if (newsCategories.length > 0) {
    try {
      newsContent = await fetchNewsFromAPI(newsCategories);
      if (newsContent.length === 0) {
        newsContent = mockNewsData.filter(item => newsCategories.includes(item.category));
      }
    } catch (error) {
      newsContent = mockNewsData.filter(item => newsCategories.includes(item.category));
    }
  }
  return [...newsContent, ...mockMovieData, ...mockSocialData];
};
```

8 UI/UX Design System, Theming & Accessibility

DESIGN SYSTEM

FeedNexus was crafted using a modern design system with strict accessibility guidelines and vibrant visual aesthetics:

🎨 HSL Color Palette

Semantic variables for primary, accent, card, muted, and destructive states.

🌙 Instant Dark Mode

Full dark mode support with Tailwind `dark:` variant classes and zero flash of unstyled content.

♿ WCAG 2.1 Compliant

High-contrast color tokens, keyboard navigation, and ARIA roles via Radix UI primitives.

Design Tokens Summary

Token Name	Light Theme Value	Dark Theme Value	Usage Description
`--background`	`hsl(0 0% 100%)` (Pure White)	`hsl(222.2 84% 4.9%)` (Deep Navy)	Main canvas background
`--card`	`hsl(0 0% 100%)`	`hsl(222.2 84% 7%)`	Content cards and modal dialogs
`--primary`	`hsl(246 80% 60%)` (Indigo)	`hsl(246 80% 60%)` (Bright Indigo)	Key buttons, active states, badges
`--border`	`hsl(214.3 31.8% 91.4%)`	`hsl(217.2 32.6% 17.5%)`	Subtle separation borders

9 Testing Strategy & Quality Assurance Matrix

QUALITY ASSURANCE

A multi-tiered test suite ensures code reliability, state correctness, and regression-free user journeys:

🧪 Unit Tests (Jest/RTL)

Validates individual reducers, actions, utility functions, and date formatters against edge cases.

🔄 Integration Tests

Verifies end-to-end data flow between the service layer, Redux store, and ContentFeed component.

🎭 E2E Tests (Cypress)

Simulates user interactions: Search input, Card Drag & Drop reordering, and Settings updates.

Edge Case Handling & Resilience Table

Potential Failure Mode	System Handling Strategy	User Experience Impact
NewsAPI Rate Limit Exceeded (429)	Automatic fallback to high-fidelity mock news dataset in `contentService.ts`.	Seamless feed without error banners.

Potential Failure Mode	System Handling Strategy	User Experience Impact
Empty Search Query Result	Renders dedicated `No results found` state with trending suggestions.	Guides user to alternate topics.
Broken Content Card Image	Shimmer skeleton with graceful fallback placeholder.	No broken image icons or layout shift.
Corrupted LocalStorage Data	`try/catch` fallback to `initialState` in store slices.	Prevents app crashes on dirty browser cache.

10 Setup, Execution & Verification Guide

OPERATIONS

> **Official Submission Repository:** `https://github.com/BinaryBlaze16/feedNexus`

1. Prerequisites

- **Node.js:** Version 18.0.0 or higher
- **Package Manager:** `npm` (v9+) or `bun` / `yarn`
- **Modern Browser:** Chrome, Safari, Firefox, or Edge

2. Installation & Quick Start

```
# 1. Clone the repository
git clone https://github.com/BinaryBlaze16/feedNexus.git

# 2. Navigate into the project directory
cd feedNexus

# 3. Install all project dependencies
npm install

# 4. Start the development server
npm run dev
```

3. NPM Scripts Reference

Command	Description
`npm run dev`	Launches Vite development server with Hot Module Replacement (HMR).
`npm run build`	Compiles and bundles the production-optimized TypeScript application.
`npm run preview`	Locally previews the production build bundle.
`npm run lint`	Runs ESLint across the codebase to enforce TypeScript code quality.

4. Verification Checklist

☑ **Feed Generation:** Content displays with dynamic images, source badges, and relative timestamps.
☑ **Category Filtering:** Changing preferences in Settings instantly updates the active feed.
☑ **Favorites Persistence:** Toggling heart icon updates Favorites tab and persists on refresh.
☑ **Drag-and-Drop:** Content cards can be dragged and repositioned smoothly.
☑ **Dark Mode:** Theme switcher updates background and persists across sessions.
☑ **Live Search:** 300ms debounced search modal returns filtered items accurately.

FeedNexus • SDE Intern Frontend Technical Submission

Engineered by Anant Srivastava • Built with React 18, TypeScript, Redux Toolkit & Tailwind CSS